Advanced Artificial Intelligence for Mechanical Engineering

Ch 1. 1) Linear Regression

Jong Moon Ha

Department of Mechanical Engineering
Ajou University

2026 Fall Semester

Course Schedule

아주대학교 AJOU UNIVERSITY

Weeks	Contents
1	1. Introduction
2-4	2. Machine Learning
5-7	3. Deep Learning
8	**Mid-term Exam**
9-10	4. Time Series
11-12	5. Generative AI
13-14	6. Industrial Issues
15	**Final Project Presentation**
16	**Final Exam**

Course Schedule

아주대학교 AJOU UNIVERSITY

Week	Thur.	Contents	Mon.	Contents
1	9/3	Introduction	9/7	No Class
2	9/10	No Class	9/14	No Class
3	9/17	ML (1) Basics	9/21	ML (1) Basics
4	9/24	추석 연휴	9/28	ML (1) Basics
5	10/1	ML (1) Basics	10/5	개천절
6	10/8	ML (2) ML Models	10/12	No Class
7	10/15	ML (2) ML Models	10/19	ML (2) ML Models
8	10/22	ML (3) Unsupervised Learning	10/26	ML (3) Unsupervised Learning
9	10/29	DL (1) Basics	11/2	DL (1) Basics
10	11/5	DL (2) CNN	11/9	DL (2) CNN
11	11/12	No Class	11/16	DL (2) CNN
12	11/19	DL (3) General Tips	11/23	DL (3) General Tips
13	11/26	Time Series (1)	11/30	Time Series (2)
14	Mid-term exam			
15	Final Project Presentation			
16	Final Project Presentation			

Machine Learning Models

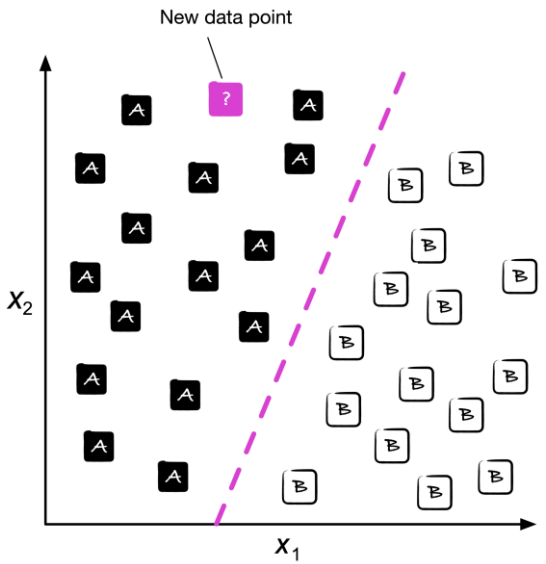
Overview

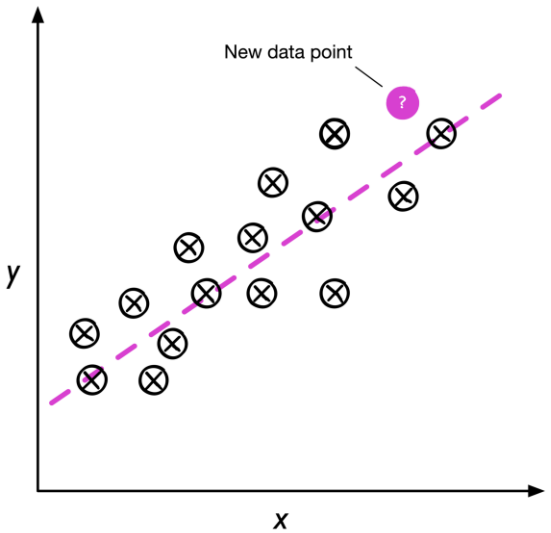
Lebel: Yes
Supervised Learning

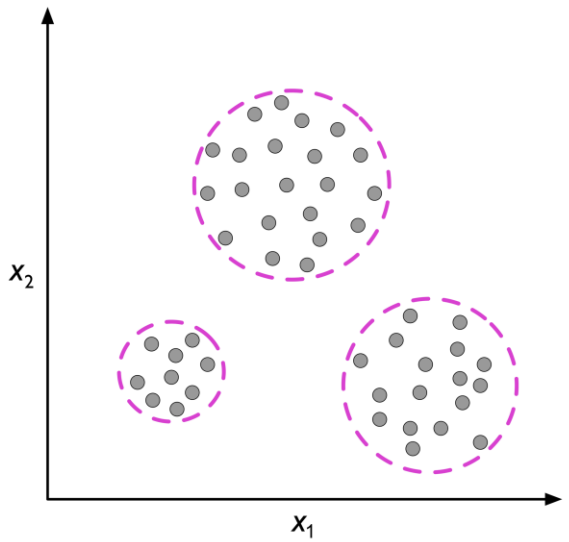
- **Discrete Label** → **Classification**
- **Continuous Label** → **Regression**

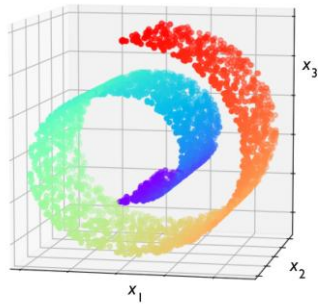
Lebel: No
Unsupervised Learning

- **Clustering**
- **Dimension reduction**









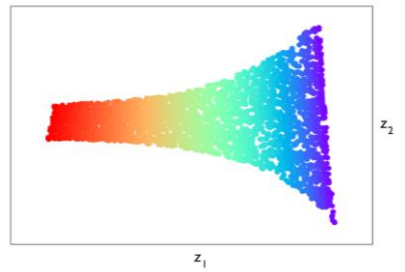
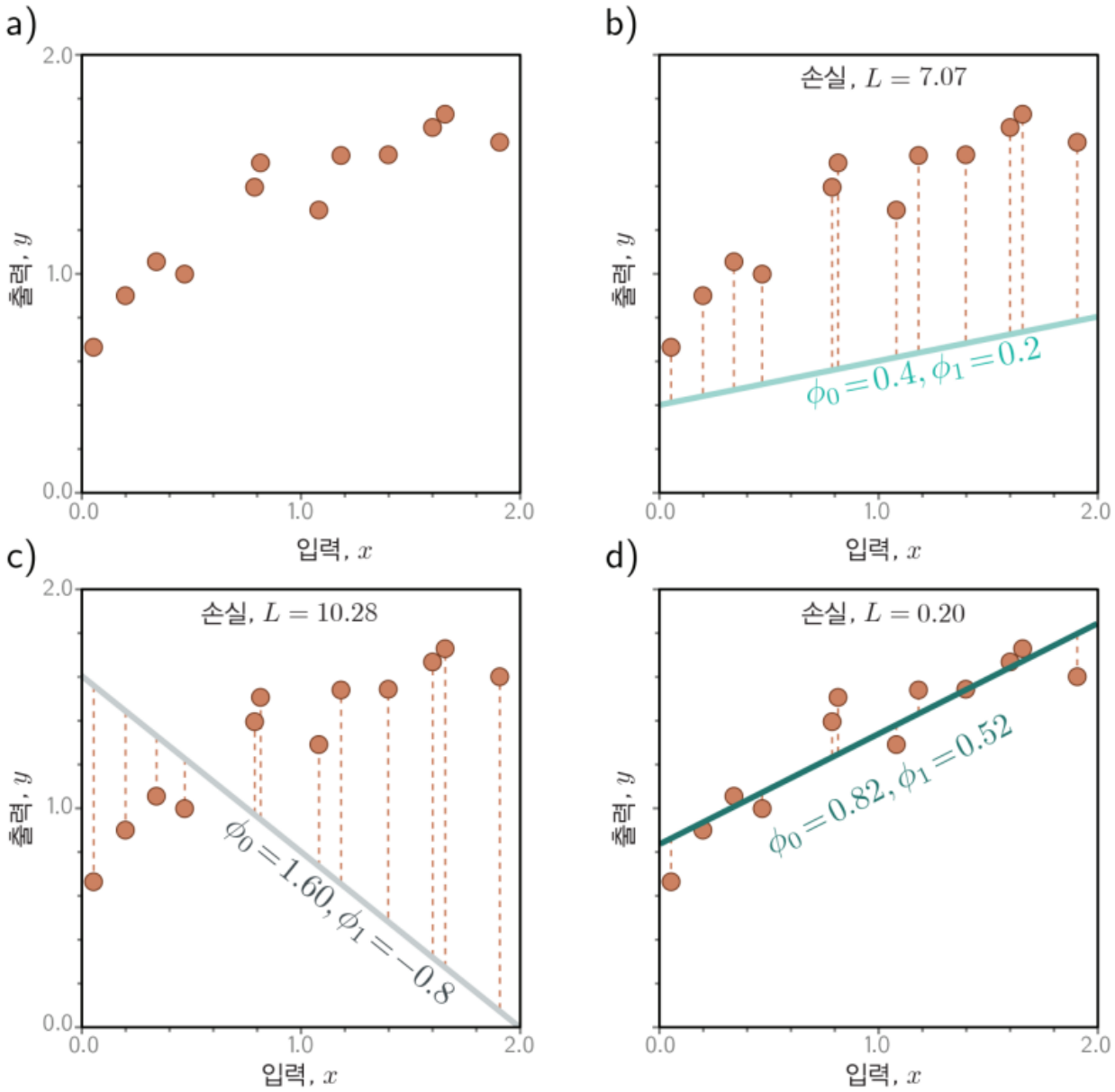


Image: Sebastian Raschka, *machine learning with pytorch and scikit-learn*

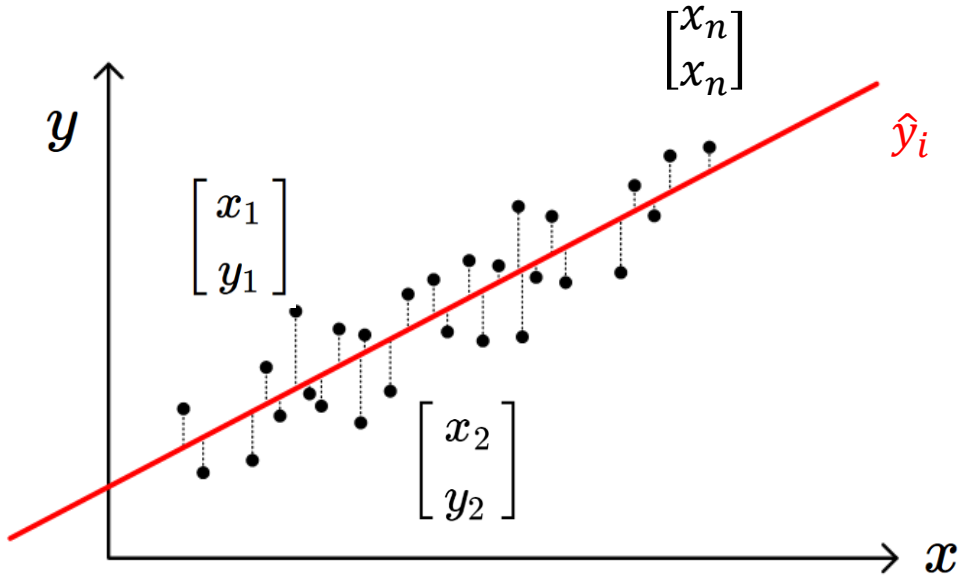
신경망 기초: 선형회귀

- 선형 회귀

  (Linear Regression)



신경망 기초: 선형회귀



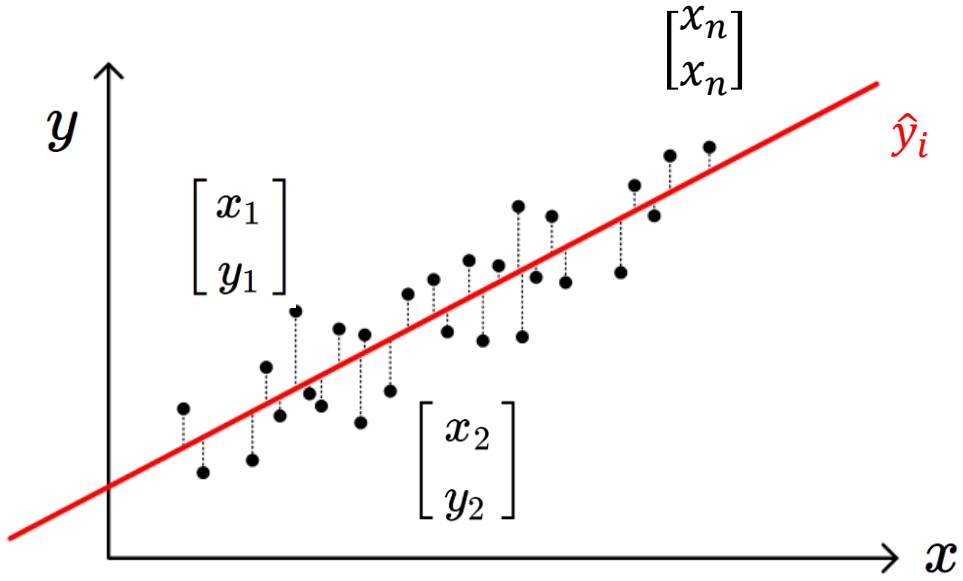
$\hat{y}_i$: predicted output at x_i by the model

$\hat{y}_n = w_0 + w_1 x_n$ $\quad$ $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$: model parameters

$\hat{y}_n = f(x_n, \mathbf{w})$ in general

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}$$

신경망 기초: 선형회귀



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \qquad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}$$

$\hat{y}_i$: predicted output at x_i by the model

$$\hat{y}_n = w_0 + w_1 x_n \qquad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}: \text{model parameters}$$

$$\hat{y}_n = f(x_n, \mathbf{w}) \text{ in general}$$

- Training the model

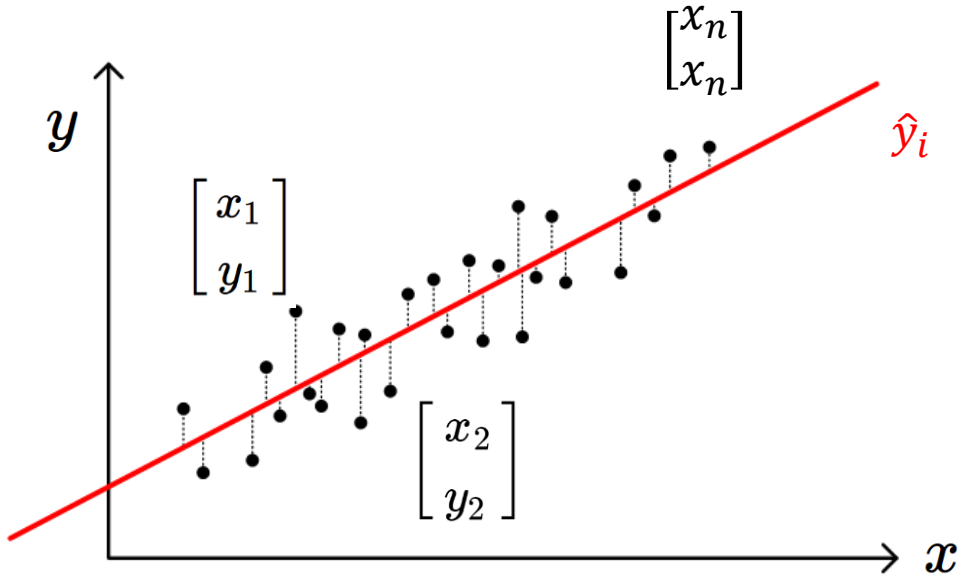
  = To find the model parameters

  by solving the optimization problem

  $=$ Find w_0 and w_1 such that $\min_{w_0, w_1} J(\mathbf{w})$

  where $J(\mathbf{w})$ is the cost function

신경망 기초: 선형회귀



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \qquad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}$$

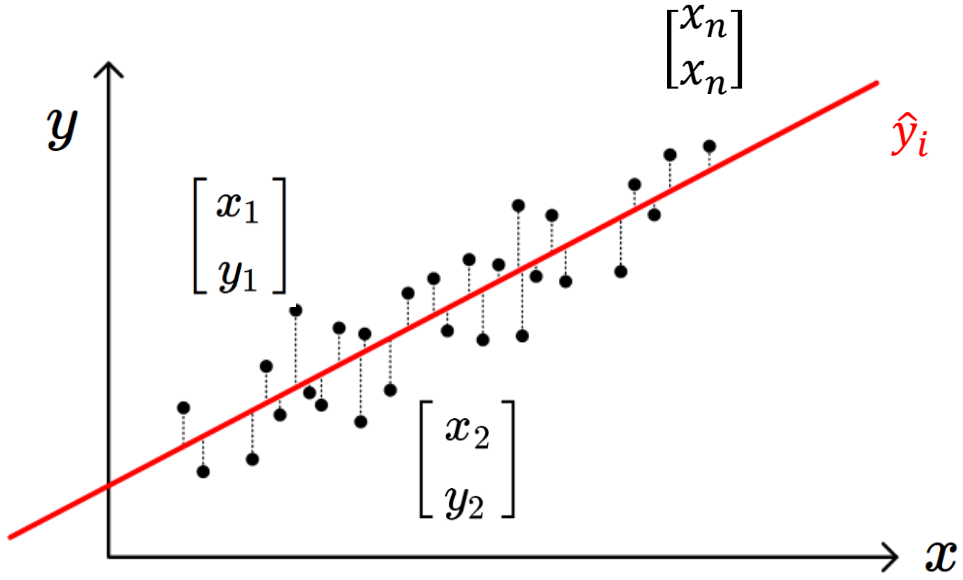
- Sample Loss (ex)

$$\ell = (\hat{y}_n - y_n)^2 \qquad \boxed{\ell = (\hat{y}_n - y_n)^2 / 2} \qquad \ell = |\hat{y}_n - y_n| \qquad \cdots$$

- Cost Function

$$J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 \qquad \boxed{J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2} \qquad J = \sum_{n=1}^{N} |\hat{y}_n - y_n| \qquad \cdots$$

신경망 기초: 선형회귀



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}$$

- Sample Loss (ex)

$$\ell = (\hat{y}_n - y_n)^2 \qquad \boxed{\ell = (\hat{y}_n - y_n)^2 / 2} \qquad \ell = |\hat{y}_n - y_n| \qquad \cdots$$

- Cost Function

$$J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 \qquad \boxed{J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2} \qquad J = \sum_{n=1}^{N} |\hat{y}_n - y_n| \qquad \cdots$$

$$= \| \hat{\mathbf{y}} - \mathbf{y} \|_2^2 / \mathbf{2}$$

Square

L2 norm

* l_p-norm of a vector

$$\|\mathbf{x}\|_p = \left(\sum_{n=1}^{N} |x_i|^p \right)^{\frac{1}{p}}$$

* Square of l_2-norm

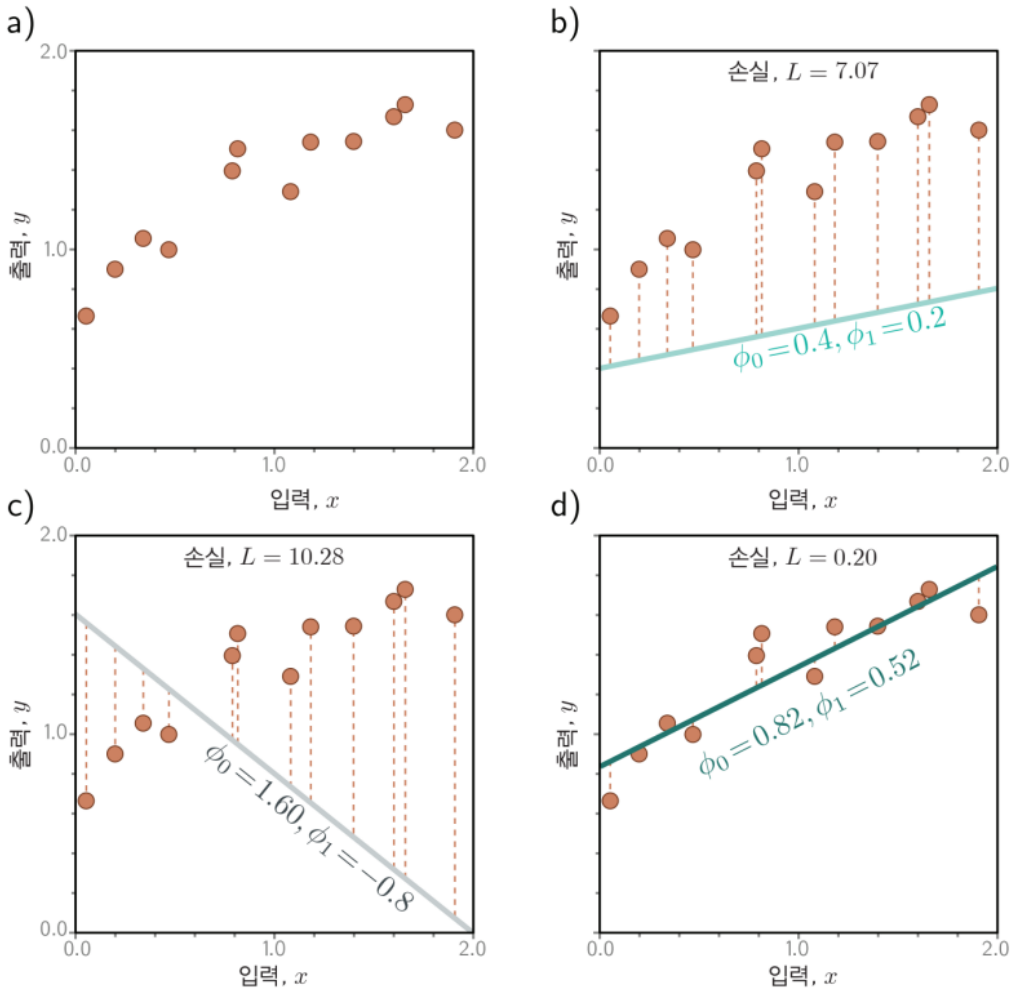
$$\|\mathbf{x}\|_2^2 = \left(\sum_{n=1}^{N} |x_i|^2 \right)$$

Square

L2 norm

신경망 기초: 선형회귀

- Training the model = To find the model parameters, **by solving the optimization problem**



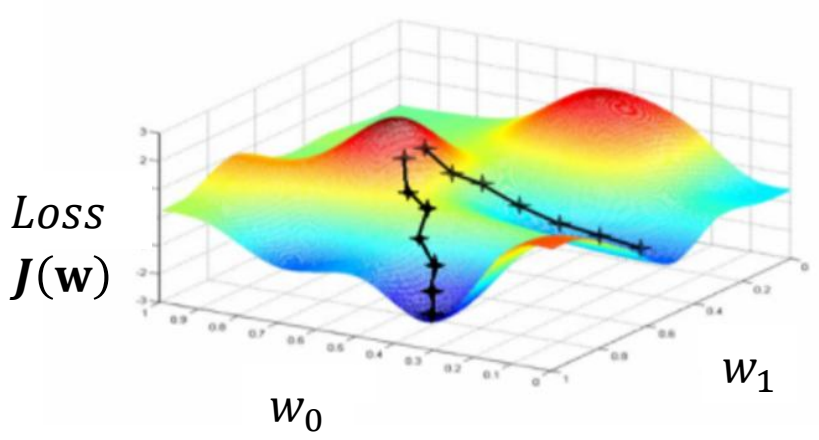
Optimization in Statistics

- **Definition**

  The process of **adjusting parameters** within a model to either **minimize or maximize a specific objective function**.

- **Key components of optimization problem**

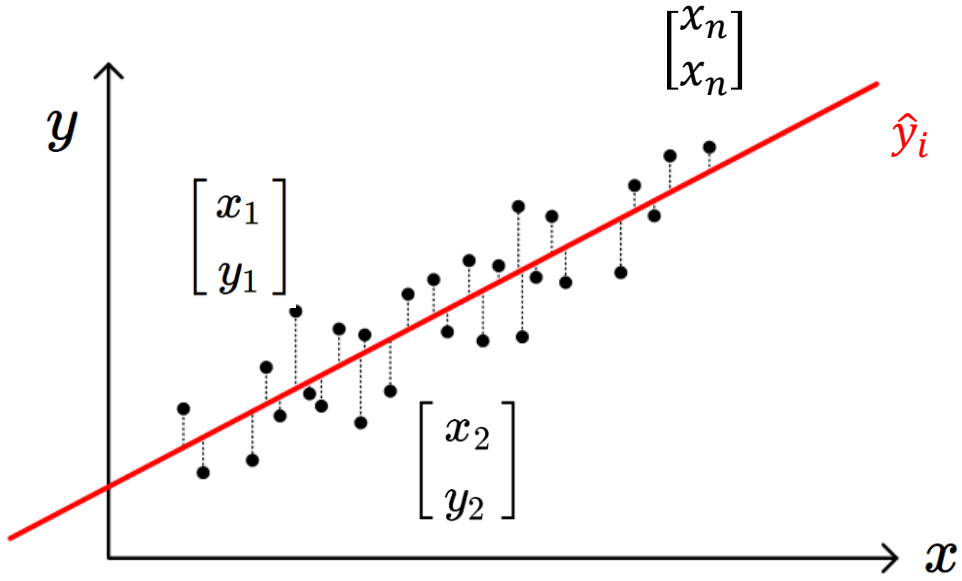
  1) Objective function (→ Loss)
  2) Decision variable (→ w_0 and w_1)
  3) Constraints (Optional)



Find $\mathbf{w}^*$ to minimize $\boldsymbol{J}(\mathbf{w})$, i.e., $\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w})$

신경망 기초: 선형회귀

$$\hat{y}_n = w_0 + w_1 x_n$$



- Training the model (Revisited)

Find w_0 and w_1 such that $\min_{w_0, w_1} J(\mathbf{w})$

where $J(\mathbf{w}) = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2$

$$\hat{y}_n = w_0 + x_n w_1 = 1 \cdot w_0 + x_n w_1$$

$$= \begin{bmatrix} 1 & x_n \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

$$= \mathbf{x}_n \, \mathbf{w}$$

Feature vector $\mathbf{x}_n = \begin{bmatrix} 1 & x_n \end{bmatrix}$

Model parameters $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

(Nx1) (Nx1)

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \Longrightarrow \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \mathbf{X}\mathbf{w}$$

신경망 기초: 선형회귀

- 최소 제곱법 (least square method)

$$J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2/2 \quad = \| \mathbf{Xw} - \mathbf{y} \|_2^2/2$$

Square

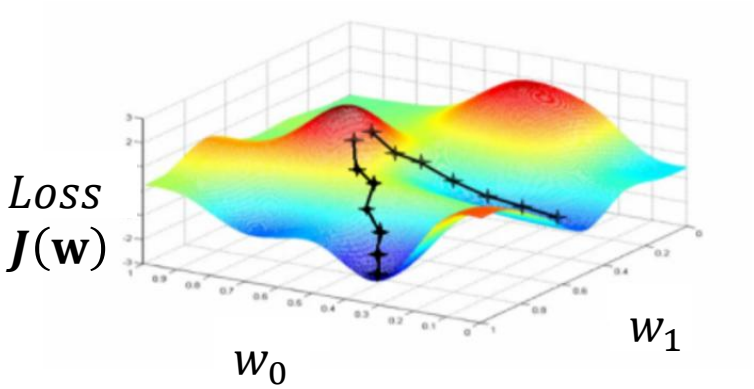
L2 norm

$$J(\mathbf{w}) = (\mathbf{Xw} - \mathbf{y})^T(\mathbf{Xw} - \mathbf{y})/2$$

$$= (\mathbf{w}^T\mathbf{X}^T - \mathbf{y}^T)(\mathbf{Xw} - \mathbf{y})/2$$

$$= (\mathbf{w}^T\mathbf{X}^T\mathbf{Xw} - \mathbf{w}^T\mathbf{X}^T\mathbf{y} - \mathbf{y}^T\mathbf{Xw} + \mathbf{y}^T\mathbf{y})/2$$

$$\frac{\partial J}{\partial \mathbf{w}} = (\mathbf{X}^T\mathbf{Xw} - \mathbf{X}^T\mathbf{y})$$



$$\frac{\partial J}{\partial \mathbf{w}} = (\mathbf{X}^T\mathbf{Xw} - \mathbf{X}^T\mathbf{y})$$

$$= -\mathbf{X}^T(\mathbf{y} - \hat{\mathbf{y}})$$

Error

← Conventional Form of Gradient of Linear Regression

The loss function is minimized when $\frac{\partial J}{\partial \mathbf{w}} = 0$

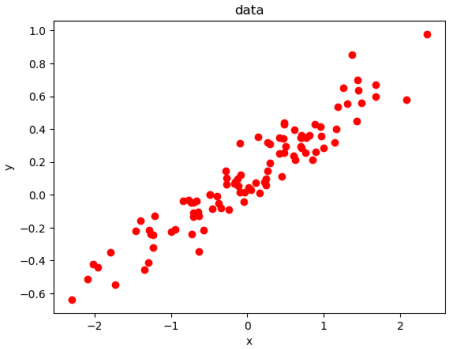
$$\frac{\partial J}{\partial \mathbf{w}} = (\mathbf{X}^T\mathbf{Xw} - \mathbf{X}^T\mathbf{y}) = \mathbf{0}$$

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

신경망 기초: 선형회귀

`y = 0.1 + 0.3 * x + noise`

`N=100`



- 최소 제곱법 (least square method) Summary

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

$$J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2$$
$$= \| \hat{\mathbf{y}} - \mathbf{y} \|_2^2 / 2$$
$$= \| \mathbf{X}\mathbf{w} - \mathbf{y} \|_2^2 / 2$$

```
# Augmented 행렬 X 구성: [1, x]
X = np.hstack([x**0, x])


# 정규방정식: w* = (XᵀX)⁻¹ Xᵀy
# 여기서 np.linalg.inv()는 역행렬을 구하는 함수.
# 그냥 (XᵀX)⁻¹로 쓸 수는 없음.
w_ls = np.linalg.inv(X.T @ X) @ X.T @ y
```

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(Nx1) (Nx2) (Nx1) (2x1)

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(Nx1) (Nx2) (2x1)

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

(Nx2) (2xN)(Nx2) (2xN)(Nx1)

신경망 기초: 선형회귀

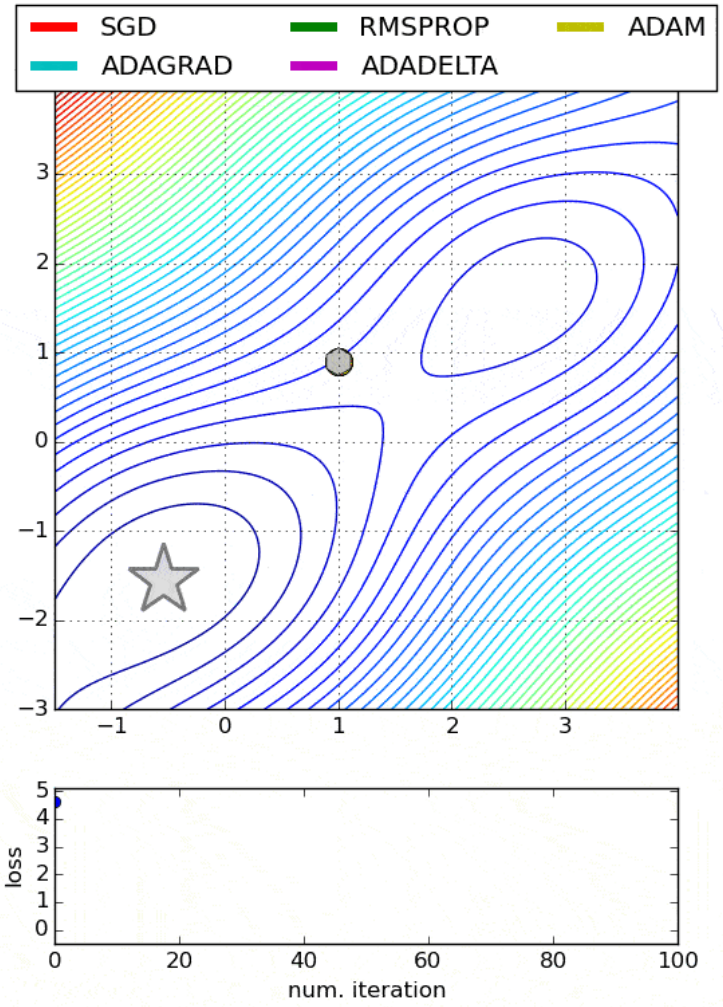
- Training the model (Revisited)

$$\hat{y}_n = w_0 + w_1 x_n \quad \text{Find } w_0 \text{ and } w_1 \text{ such that } \min_{w_0, w_1} J(\mathbf{w}) \quad \text{where } J(\mathbf{w}) = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2$$

What if we cannot derive $\mathbf{w}^*$ when the model is complicated?

➔ We need "Iterative Optimization"

Various Iterative Optimization Algorithms

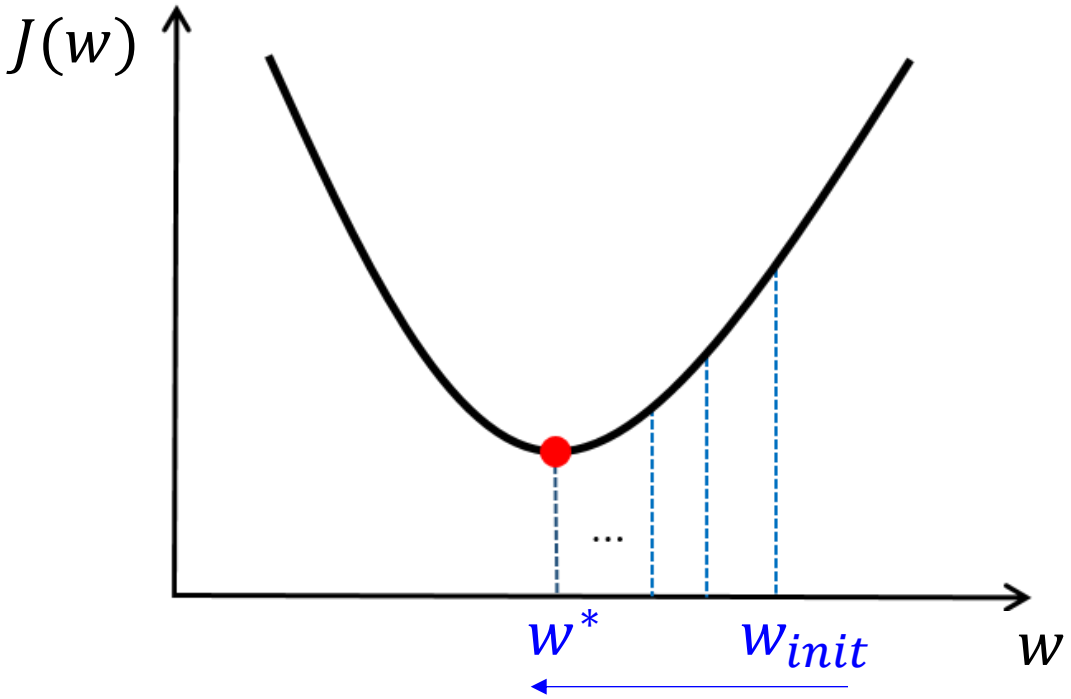


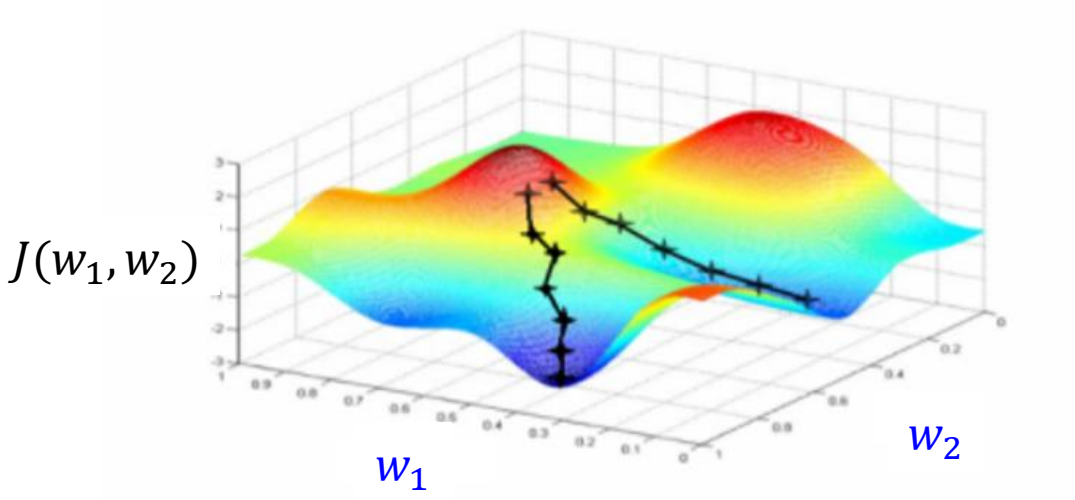
Gradient Descent

$J'(w) = \frac{\partial J}{\partial w}$ **is gradient of the function ➔ Indicate increasing direction**

➔ We can minimize J(w) by moving in $-\frac{\partial J}{\partial w}$ direction

Repeat: $w \leftarrow w - \rho \frac{\partial J}{\partial w}$ for some step size $\rho > 0$





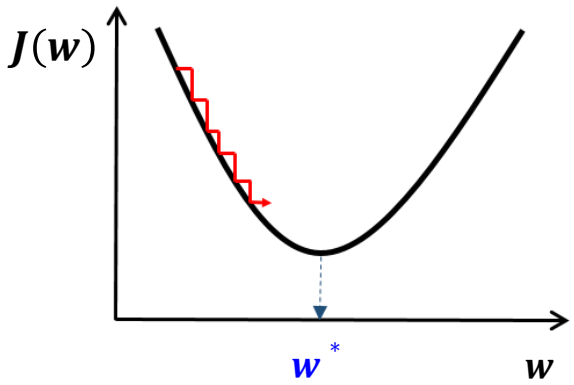
Gradient Descent

$J'(w) = \frac{\partial J}{\partial w}$ **is gradient of the function ➔ Indicate increasing direction**

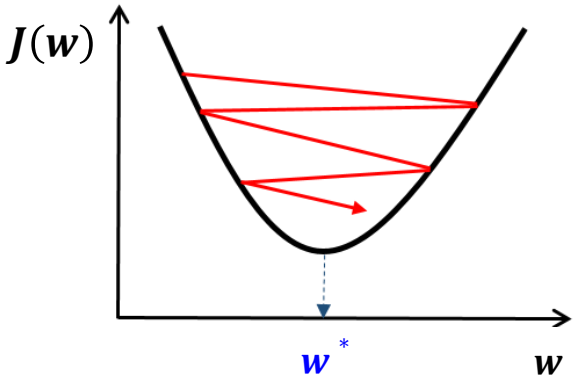
➔ We can minimize J(w) by moving in $-\frac{\partial J}{\partial w}$ **direction**

Repeat: $w \leftarrow w - \rho \frac{\partial J}{\partial w}$ for some step size $\rho > 0$

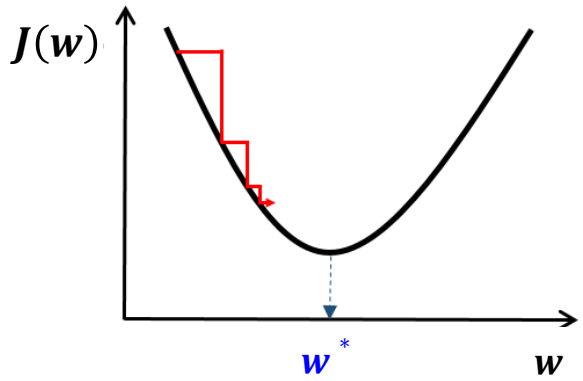
- **Choosing step size ρ (learning rate)**



Too small: converge very slowly



Too big: overshoot and even diverge



Reduce size over time

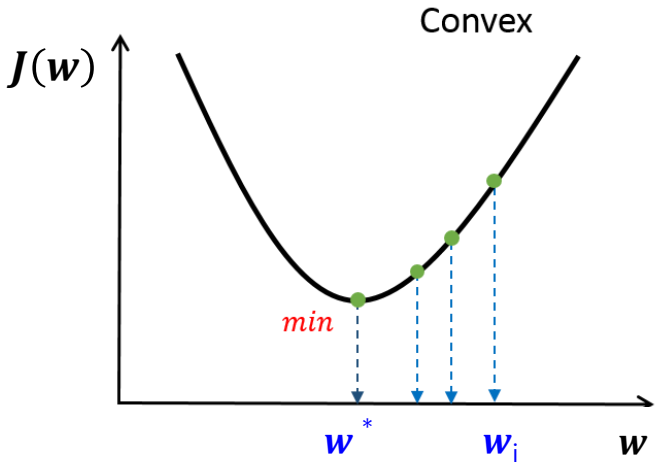
Gradient Descent

$J'(w) = \frac{\partial J}{\partial w}$ **is gradient of the function ➔ Indicate increasing direction**

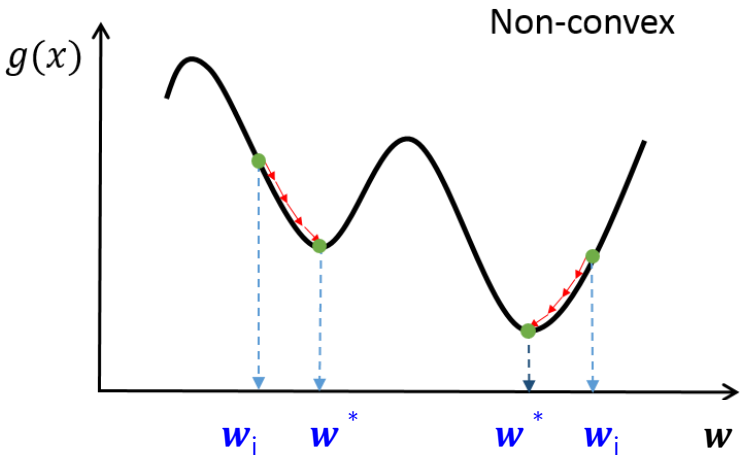
➔ We can minimize J(w) by moving in $-\frac{\partial J}{\partial w}$ direction

Repeat: $w \leftarrow w - \rho \frac{\partial J}{\partial w}$ for some step size $\rho > 0$

- **Global minimum & Local minima**
  - Random initialization
  - Multiple trials



Any local minimum is a global minimum



Multiple local minima may exist

신경망 기초: 선형회귀

- Training the model (Revisited)

$\hat{y}_n = w_0 + w_1 x_n$ Find w_0 and w_1 such that $\min_{w_0, w_1} J(\mathbf{w})$ where $J(\mathbf{w}) = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2$

Least Square Method

$$J(\mathbf{w}) = (\mathbf{Xw} - \mathbf{y})^T (\mathbf{Xw} - \mathbf{y})/2$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T\mathbf{Xw} - \mathbf{X}^T\mathbf{y} = 0$$

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Gradient Decent Method

$$J(\mathbf{w}) = (\mathbf{Xw} - \mathbf{y})^T (\mathbf{Xw} - \mathbf{y})/2$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T\mathbf{Xw} - \mathbf{X}^T\mathbf{y} = 0$$

$$= \mathbf{X}^T(\mathbf{Xw} - \mathbf{y}) = 0$$

$$= \mathbf{X}^T(\hat{\mathbf{y}} - \mathbf{y}) = 0$$

Repeat: $\mathbf{w} \leftarrow \mathbf{w} - \rho \frac{\partial J}{\partial \mathbf{w}}$

Gradient Decent Process

1) Compute $\hat{\mathbf{y}} = \mathbf{Xw}$
2) Compute $\mathbf{E} = \hat{\mathbf{y}} - \mathbf{y}$
3) Compute $\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T\mathbf{E}$

Repeat: $\mathbf{w} \leftarrow \mathbf{w} - \rho \frac{\partial J}{\partial \mathbf{w}}$

신경망 기초: 선형회귀

- Training the model (Revisited)

$$\hat{y}_n = w_0 + w_1 x_n \quad \text{Find } w_0 \text{ and } w_1 \text{ such that } \min_{w_0, w_1} J(\mathbf{w}) \quad \text{where } J(\mathbf{w}) = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2$$

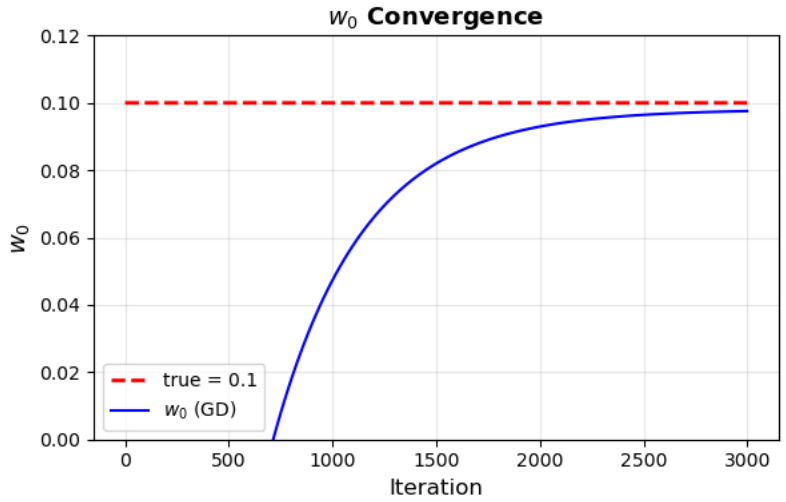
```
# 초기 파라미터 (랜덤)
w = np.random.randn(2, 1)

# 하이퍼파라미터
rho = 0.00002        # step size
n_iter = 3000        # 반복 횟수
w_history = []

# 경사하강법 반복
for i in range(n_iter):
    y_hat = X @ w                          # ŷ = Xw
    E = y - y_hat                          # 잔차 E = y - ŷ
    dJ = -X.T @ E                          # ∂J/∂w = -Xᵀ E
    w  = w - rho * dJ                      # w ← w - ρ·∂J/∂w
    w_history.append(w[:, 0])

    if i % 500 == 0 or i == n_iter - 1:
        loss = float(np.sum(E**2))         # J = ‖y - ŷ‖²
        print(f"iter={i:5d}  w₀={w[0,0]:+.6f}  w₁={w[1,0]:+.6f}  J={loss:.6f}")

# y_hat (예측값)
y_hat = X @ w
```

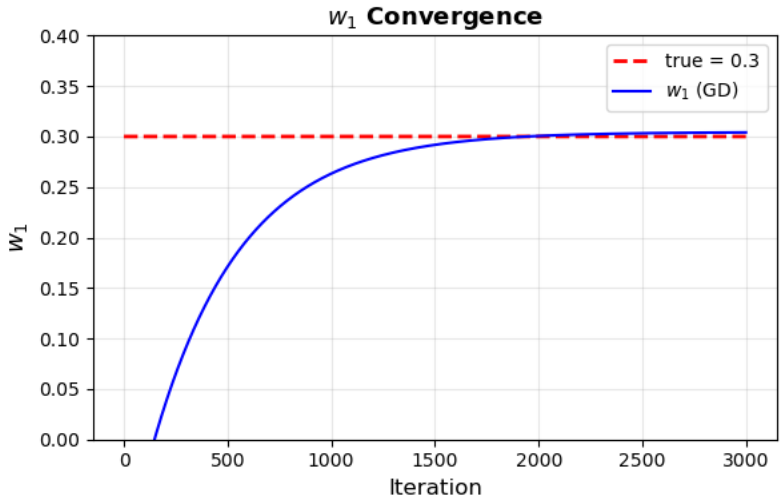
Gradient Decent Process

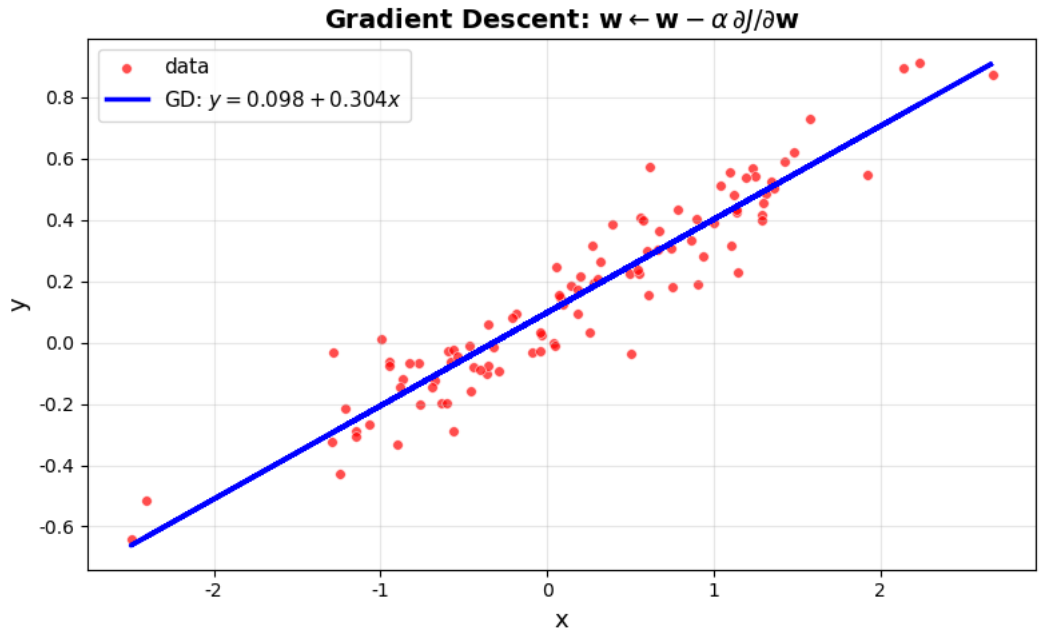
1) Compute $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$
2) Compute $\mathbf{E} = \hat{\mathbf{y}} - \mathbf{y}$
3) Compute $\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T \mathbf{E}$

Repeat: $\mathbf{w} \leftarrow \mathbf{w} - \rho \frac{\partial J}{\partial \mathbf{w}}$

경사하강법 (수식적 풀이)


w_0 Convergence
0.12
0.10
0.08
0.06
0.04
0.02
0.00
w_0
true = 0.1
w_0 (GD)
0
500
1000
1500
2000
2500
3000
Iteration


w_1 Convergence
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
w_1
true = 0.3
w_1 (GD)
0
500
1000
1500
2000
2500
3000
Iteration


Gradient Descent: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\, \partial J/\partial \mathbf{w}$
data
GD: $y = 0.098 + 0.304x$
0.8
0.6
0.4
0.2
0.0
-0.2
-0.4
-0.6
y
-2
-1
0
1
2
x

경사하강법 (수식적 풀이)

$\hat{y}_n = w_0 + w_1 x_n$ Find w_0 and w_1 such that $\min_{w_0, w_1} J(\mathbf{w})$ where $J(\mathbf{w}) = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 / 2$

What if the derivative of the loss function is not well-defined or unknown?

(∇f in this case)

Auto Gradient using PyTorch

Gradient Decent Method

$$J(\mathbf{w}) = (\mathbf{Xw} - \mathbf{y})^T (\mathbf{Xw} - \mathbf{y})/2$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T\mathbf{Xw} - \mathbf{X}^T\mathbf{y} = 0$$

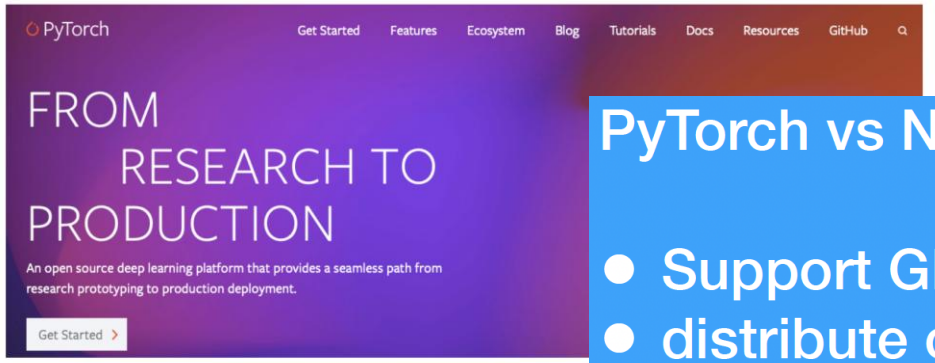
$$= \mathbf{X}^T(\mathbf{Xw} - \mathbf{y}) = 0$$

$$= \mathbf{X}^T(\hat{\mathbf{y}} - \mathbf{y}) = 0$$

Repeat: $w \leftarrow w - \rho \frac{\partial J}{\partial \mathbf{w}}$

Intro. to Pytorch

PyTorch



https://pytorch.org/

PyTorch vs NumPy

- Support GPU
- distribute ops across multiple devices
- keep track of computation graph and ops that created them

```
1  # tensor 생성 (requires_grad=True → autograd 활성화)
2  x = torch.tensor(2.0, requires_grad=True)
3  print('x:', x)
4  print('requires_grad:', x.requires_grad)
```

✓ 0.0s

```
x: tensor(2., requires_grad=True)
requires_grad: True
```

References:
https://pytorch.org/
https://docs.pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html

Gradient with Pytorch ex 1)

$$y = w^2 + 2w + 1$$

$$\frac{dy}{dw} = ?$$

Let

$w = 2$ $\quad y = 9$

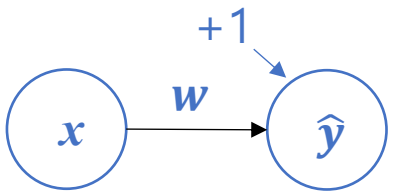
$$\frac{dy}{dw} = 2w + 2$$

$$\frac{dy}{dw} = 6$$

```
w = 2 # Try different values
w_tensor = torch.tensor(w, dtype=float,
                        requires_grad=True)
y_tensor = w_tensor**2 + 2*w_tensor + 1

# calculate the gradient
y_tensor.backward()

print('w: ', w_tensor)
print('y: ', y_tensor)
print('grad: ', w_tensor.grad)    # 2*2+2 = 6
```

```
w:  tensor(2., dtype=torch.float64, requires_grad=True)
y:  tensor(9., dtype=torch.float64, grad_fn=<AddBackward0>)
grad:  tensor(6., dtype=torch.float64)
```

Gradient with Pytorch ex 2)



$$\hat{y} = xw + 1$$

$\frac{d\hat{y}}{dw} = ?$

Let

$x = 3$

$w = 2$

$\hat{y} = 7$

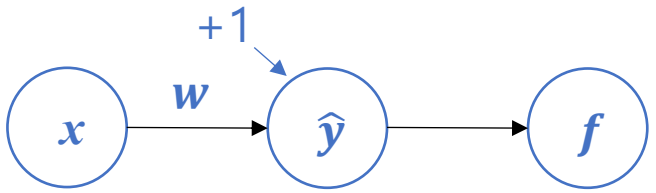
$$\frac{d\hat{y}}{dw} = x$$

$\frac{d\hat{y}}{dw} = 3$

```
w = 2 # Try different values
w_tensor = torch.tensor(w, dtype=float,
                        requires_grad=True)
y_tensor = 3*w_tensor + 1

# calculate the gradient
y_tensor.backward()

print('w: ', w_tensor)
print('y: ', y_tensor)
print('grad: ', w_tensor.grad)
```

```
w:  tensor(2., dtype=torch.float64, requires_grad=True)
y:  tensor(7., dtype=torch.float64, grad_fn=<AddBackward0>)
grad:  tensor(3., dtype=torch.float64)
```

Gradient with Pytorch ex 3)



$\hat{y} = xw + 1$ $\quad f = \hat{y}^2$

$\frac{df}{dw} = ?$

Let

$x = 3$ $\quad \hat{y} = 7$ $\quad f = 49$

$w = 2$

$\frac{d\hat{y}}{dw} = x$ $\quad \frac{df}{d\hat{y}} = 2\hat{y}$ $\quad \frac{df}{dw} = \frac{dy}{dw}\frac{df}{dy} = 2x\hat{y}$

$\frac{dy}{dw} = 3$ $\quad \frac{df}{dy} = 14$ $\quad \frac{df}{dw} = \frac{dy}{dw}\frac{df}{dy} = 42$

```
w = 2 # Try different values
w_tensor = torch.tensor(w, dtype=float,
                        requires_grad=True)
y_tensor = 3*w_tensor + 1
f = y_tensor**2

# calculate the gradient
f.backward()

print('w: ', w_tensor)
print('y: ', y_tensor)
print('f: ', f)
print('df/dw: ', w_tensor.grad)
```

```
w:  tensor(2., dtype=torch.float64, requires_grad=True)
y:  tensor(7., dtype=torch.float64, grad_fn=<AddBackward0>)
f:  tensor(49., dtype=torch.float64, grad_fn=<PowBackward0>)
df/dw:  tensor(42., dtype=torch.float64)
```

Gradient Descent with Pytorch

```
1  # parameters to optimize
2  # Initial value is randomly set
3  # requ
4
5  def fi
6      t0                                                    ue)
7      t1                                                    ue)
8      op
9
10     #
11     th
12     fo
13
14
15
16
17
18                                                           dated)
19
20
21
22
23
24
25                                                    {:3f}'\
26
27     theta_history = np.array(theta_history)
28
29     return theta_history, t0, t1
```

Revision required

Gradient Decent Method

$$J(\mathbf{w}) = (\mathbf{Xw} - \mathbf{y})^T(\mathbf{Xw} - \mathbf{y})/2$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T\mathbf{Xw} - \mathbf{X}^T\mathbf{y} = 0$$

$$= \mathbf{X}^T(\mathbf{Xw} - \mathbf{y}) = 0$$

$$= \mathbf{X}^T(\hat{\mathbf{y}} - \mathbf{y}) = 0$$

Repeat: $w \leftarrow w - \rho \frac{\partial J}{\partial \mathbf{w}}$

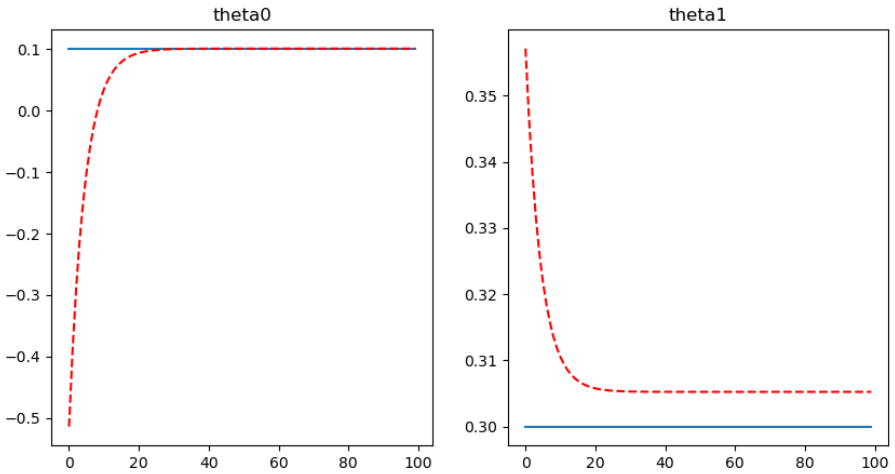
Gradient Descent with Pytorch

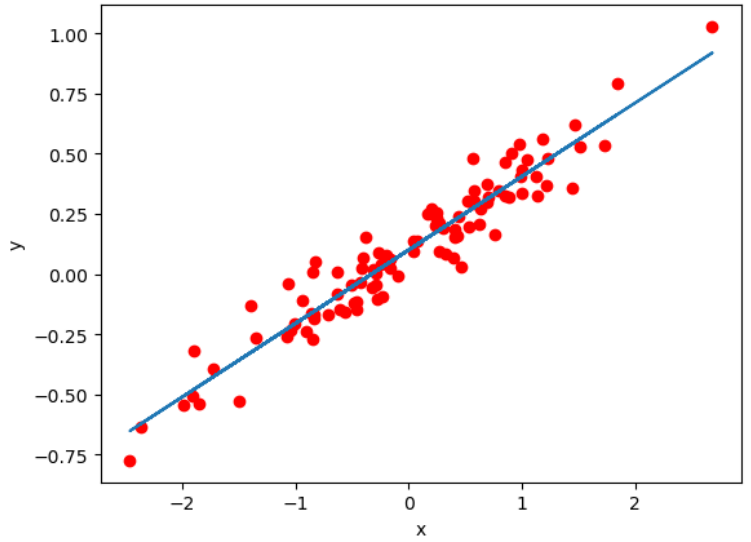
```
1  n_iter = 100
2  step_size
3
4  theta_hist
5
```

✓ 0.6s Python

Revision required

```
iter= 0,
iter= 10,
iter= 20,
iter= 30,
iter= 40,
iter= 50,
iter= 60,
iter= 70,
iter= 80,   theta0: 0.104085,   theta1: 0.302501,   loss: 0.909886
iter= 90,   theta0: 0.104085,   theta1: 0.302502,   loss: 0.909886
```





Linear Regression with Pytorch

```
######################################################################
# Investigate the manual gradient descent step
# Code withi                                                    mizer
# Gradient u
theta0_grad
theta1_grad
t0_manual1 =
t1_manual1 =

# Gradient u
theta_math =
df = 2 * (A.
theta_math =
t0_manual2 =
t1_manual2 =
#############
```

Revision required

Torch step: optimizer.step()

Manual 1: $x \leftarrow x - \alpha \nabla_x f(x)$

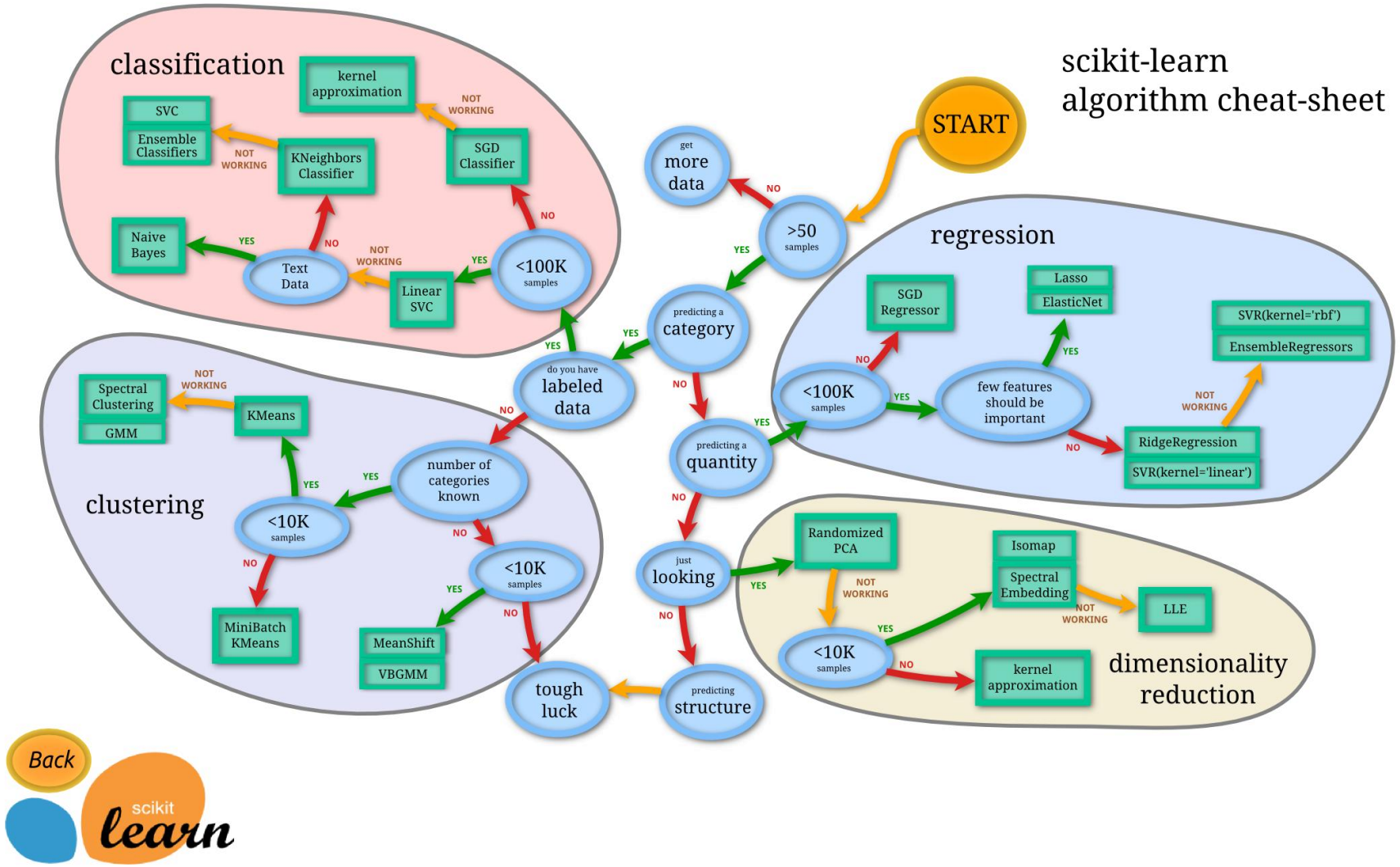
∇f from torch.grad

Manual 2: $x \leftarrow x - \alpha \nabla_x f(x)$

$\nabla f = 2(\Phi^T \Phi \theta - \Phi^T y)$

```
iter= 0,                                                    l,  t1 (torch step, manual 1, 2): -0.136128, -0.136128, -0.136128,
iter= 10,                                                   3,  t1 (torch step, manual 1, 2): 0.243209, 0.243209, 0.243209,
iter= 20,                                                       t1 (torch step, manual 1, 2): 0.293700, 0.293700, 0.293700,
iter= 30,                                                       t1 (torch step, manual 1, 2): 0.300295, 0.300295, 0.300295,
iter= 40,                                                       t1 (torch step, manual 1, 2): 0.301145, 0.301145, 0.301145,
iter= 50,                                                       t1 (torch step, manual 1, 2): 0.301253, 0.301253, 0.301253,
iter= 60,                                                       t1 (torch step, manual 1, 2): 0.301267, 0.301267, 0.301267,
iter= 70,       t0 (torch step, manual 1, 2): 0.099812, 0.099812, 0.099812,     t1 (torch step, manual 1, 2): 0.301268, 0.301268, 0.301268,
iter= 80,       t0 (torch step, manual 1, 2): 0.099812, 0.099812, 0.099812,     t1 (torch step, manual 1, 2): 0.301268, 0.301268, 0.301268,
iter= 90,       t0 (torch step, manual 1, 2): 0.099812, 0.099812, 0.099812,     t1 (torch step, manual 1, 2): 0.301269, 0.301269, 0.301269,
```

Linear Regression with Scikit-Learn

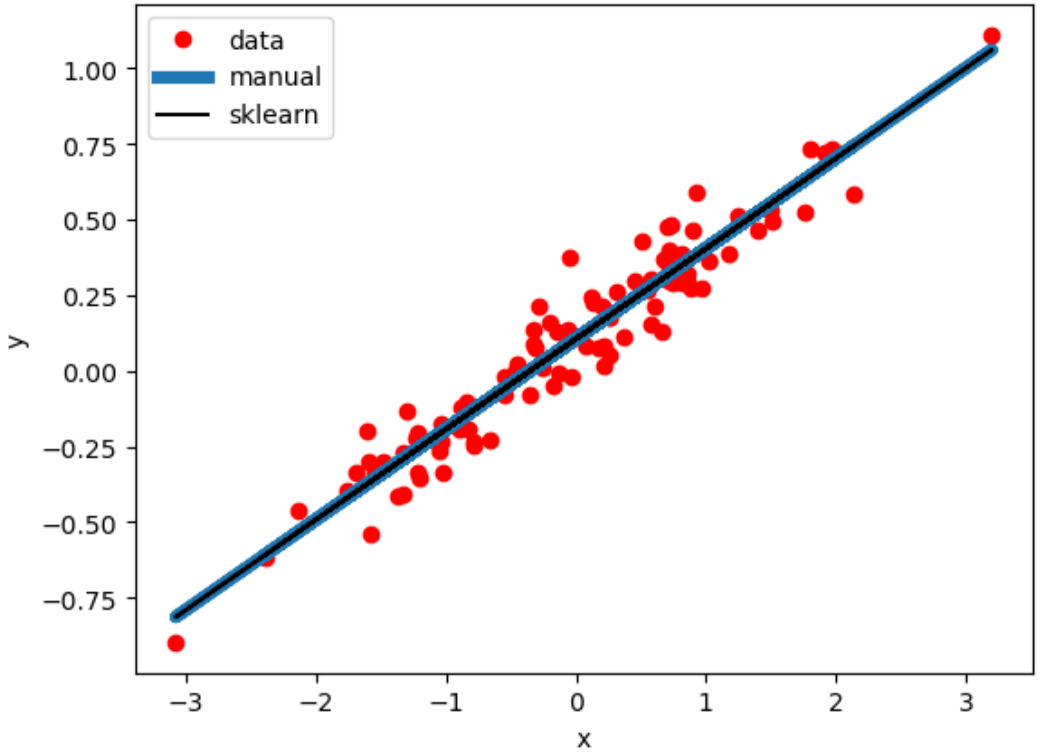


Linear Regression with Scikit-Learn

```python
from sklearn import linear_model # type: ignore
reg = linear_model.LinearRegression()
reg.fit(x, y)
print(reg.coef_)
print(reg.intercept_)

plt.figure()
plt.plot(x, y, 'ro')
plt.plot(x, reg.coef_*x + reg.intercept_, linewidth=5)
plt.plot(x, reg.predict(x), 'k')
plt.xlabel('x'), plt.ylabel('y')
plt.legend(['data', 'manual', 'sklearn'])
plt.show()
```

✓ 0.0s

```
[[0.29867749]]
[0.10550319]
```



선형회귀 정리

- 선형회귀 종합 (경사하강법) 종합

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

$$J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 = \| \hat{\mathbf{y}} - \mathbf{y} \|_2^2 = \| \mathbf{X}\mathbf{w} - \mathbf{y} \|_2^2$$

$$\frac{\partial J}{\partial \mathbf{w}} = (\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = -\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

```
# Augmented 행렬 X 구성: [1, x]
X = np.hstack([x**0, x])
```

```
# 초기 파라미터 (랜덤)
w = np.random.randn(2, 1)
```

```
y_hat = X @ w
E = y - y_hat
dJ = -X.T @ E
w  = w - rho * dJ
```

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

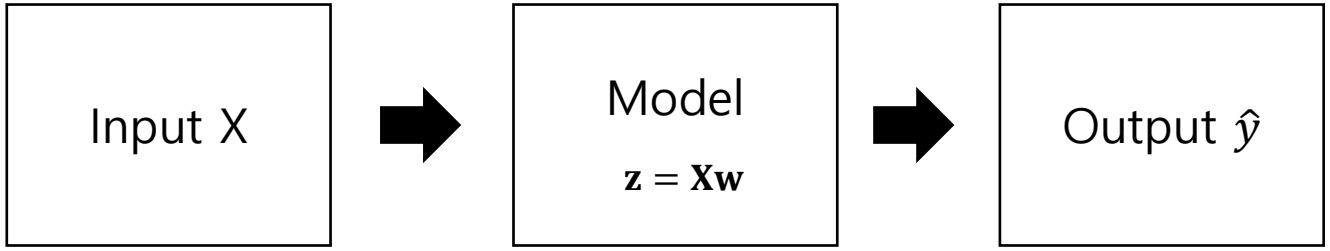
(Nx1) (Nx2) (Nx1) (2x1)

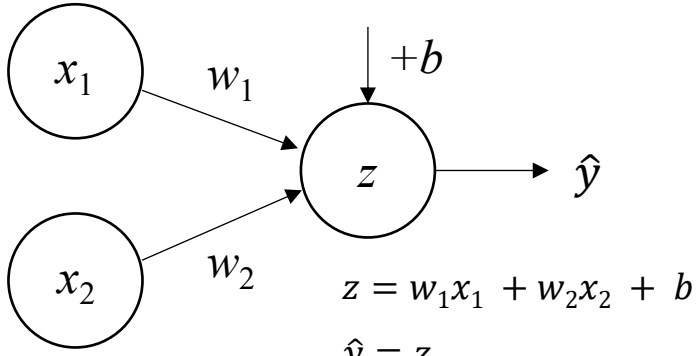
$$\underset{\text{(Nx1)}}{\hat{\mathbf{y}}} = \underset{\text{(Nx2)}}{\mathbf{X}} \underset{\text{(2x1)}}{\mathbf{w}} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

$$\underset{\text{(2x1)}}{\frac{\partial J}{\partial \mathbf{w}}} = \underset{\text{(2xN)}}{-\mathbf{X}^T} \underset{\text{(Nx1)}}{(\mathbf{y} - \mathbf{X}\mathbf{w})}$$

선형회귀 정리

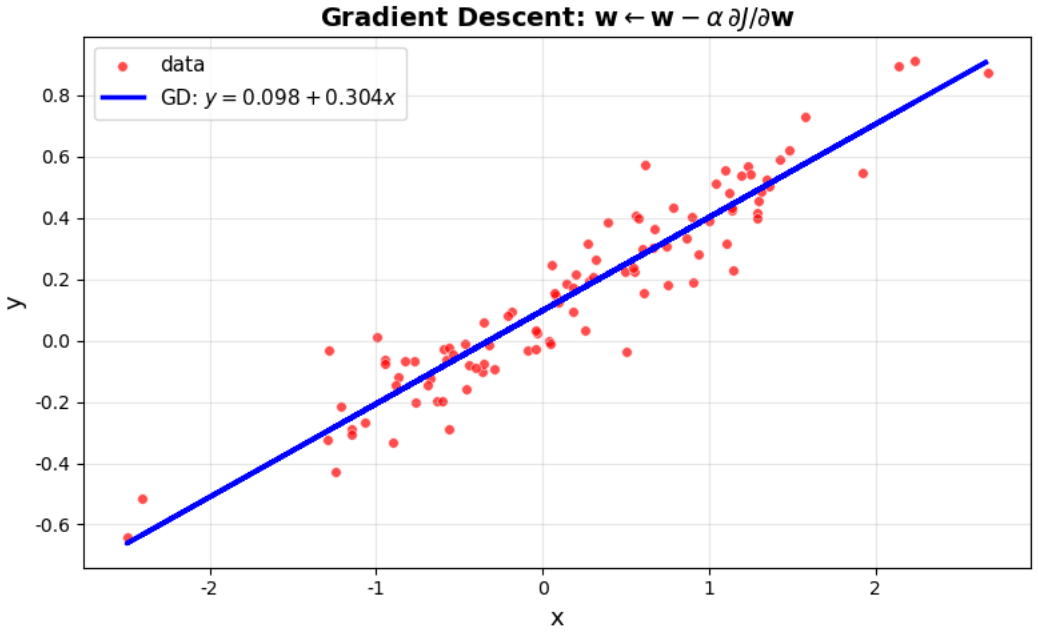
- 선형회귀 종합 (경사하강법) 종합





$$z = w_1 x_1 + w_2 x_2 + b$$

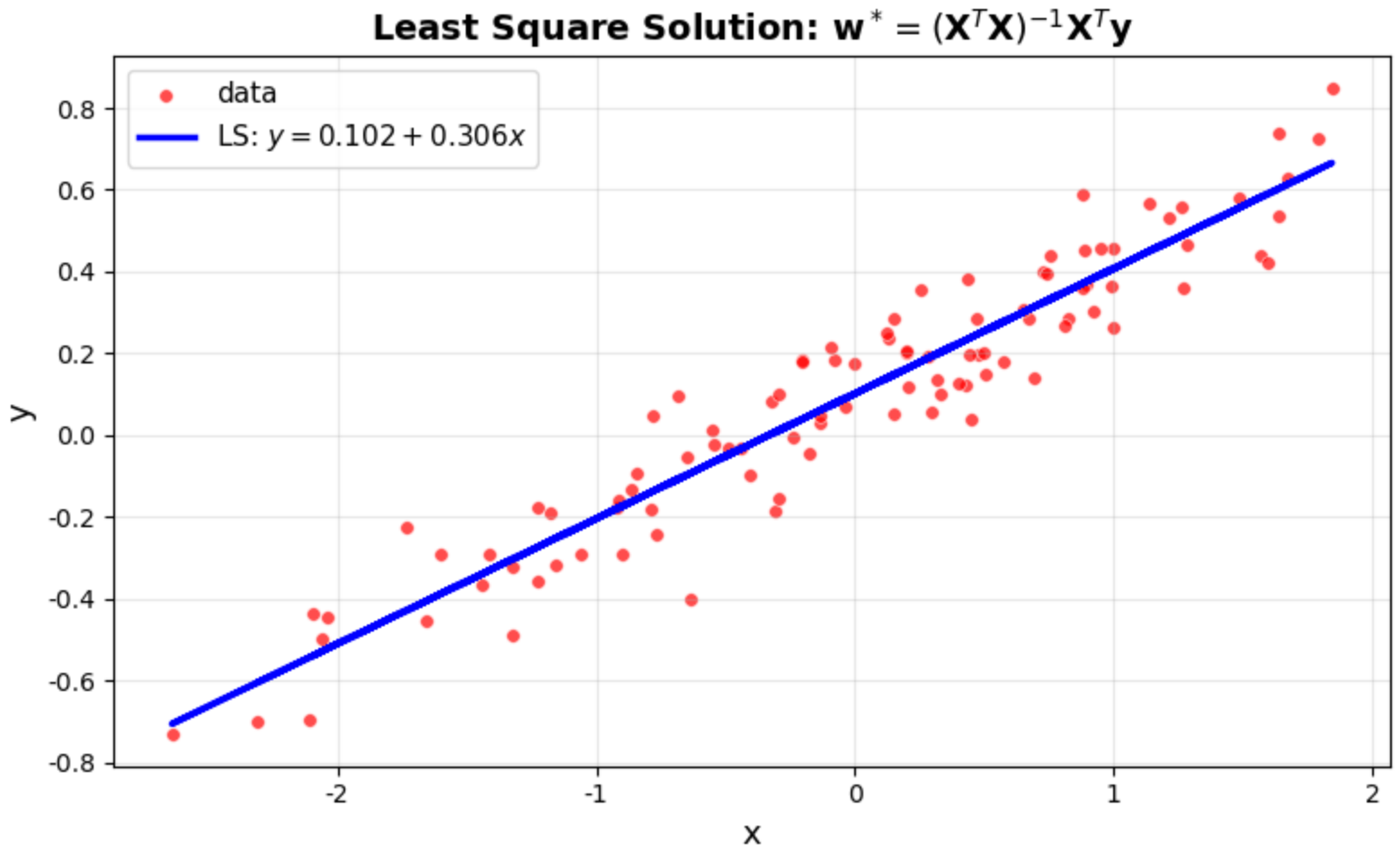
$$\hat{y} = z$$



선형 회귀 실습

Practice 02

아주대학교 AJOU UNIVERSITY

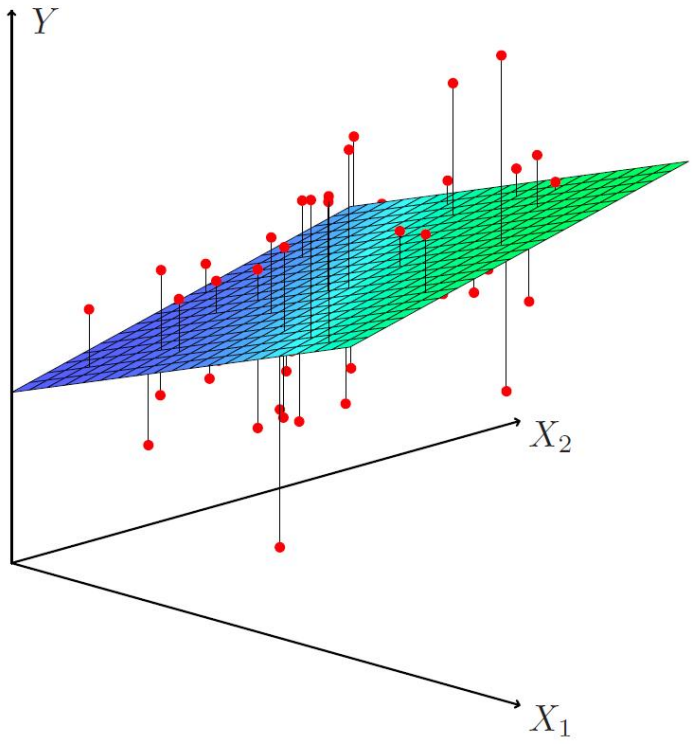


선형회귀 – Multivariate & Nonlinear (Polynomial)

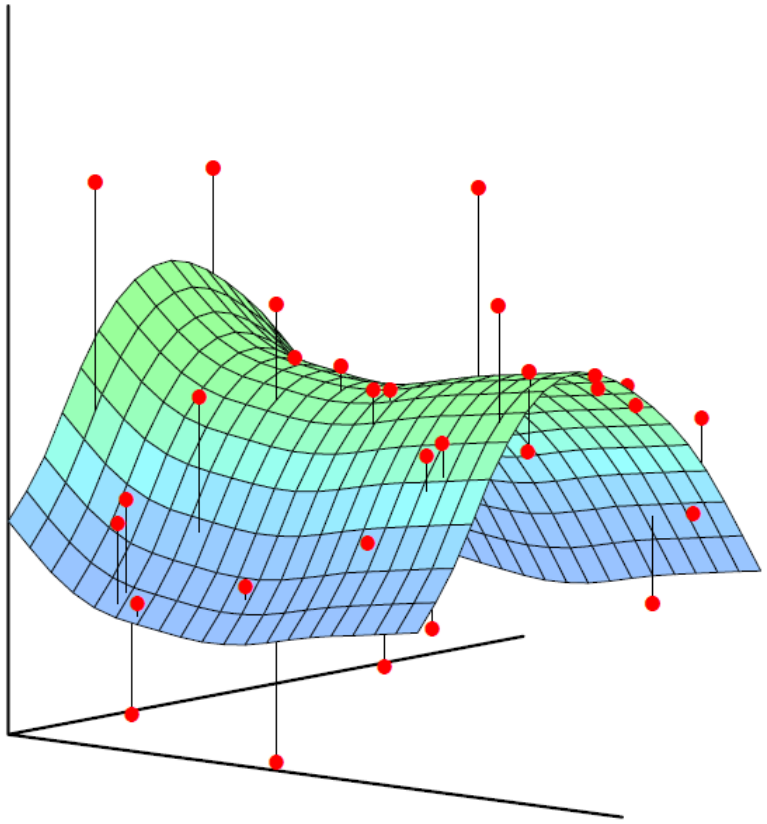
HW

아주대학교 AJOU UNIVERSITY

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2$$



$$\hat{y} = w_0 + w_1 x_1^2 + w_2 x_2^2$$



선형회귀 – Multivariate & Nonlinear (Polynomial)

HW

아주대학교 AJOU UNIVERSITY

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2$$

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_1^2$$

Feature vector

$$\mathbf{x}_n = \begin{bmatrix} 1 & x_1^{(n)} & x_2^{(n)} \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} \end{bmatrix}$$

$$\mathbf{x}_n = \begin{bmatrix} 1 & x_1^{(n)} & \left(x_1^{(n)}\right)^2 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \left(x_1^{(1)}\right)^2 \\ 1 & x_1^{(2)} & \left(x_1^{(2)}\right)^2 \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(N)} & \left(x_1^{(N)}\right)^2 \end{bmatrix}$$

Model parameters

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

$$\hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix} = \mathbf{X}\mathbf{w}$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

➔ **Actually linear regression**
to solve the multi-variable / nonlinear problem

Any Questions?

선형회귀 – 벡터를 표현하는 배열의 형태에 따라

아주대학교 AJOU UNIVERSITY

■ 선형 회귀

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

$$J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 = \| \hat{\mathbf{y}} - \mathbf{y} \|_2^2 = \| \mathbf{X}\mathbf{w} - \mathbf{y} \|_2^2$$

$$\frac{\partial J}{\partial \mathbf{w}} = (\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = -\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

```
# Augmented 행렬 X 구성: [1, x]
X = np.hstack([x**0, x])
```

```
# 초기 파라미터 (랜덤)
w = np.random.randn(2, 1)
```

```
y_hat = X @ w
E = y - y_hat
dJ = -X.T @ E
w  = w - rho * dJ
```

벡터를 2D 배열로 표현

$$\hat{y} = w_0x_0 + w_1x_1 + w_2x_2$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(Nx1) (Nx2) (Nx1) (2x1)

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(Nx2) (2x1) → (Nx1) (Nx2) (2x1)

$$\frac{\partial J}{\partial \mathbf{w}} = -\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

(Nx1) (Nx2)(2x1) → (2x1) (2xN) (Nx1)

벡터를 1D 배열로 표현

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(N,) (Nx2) (N,) (2,)

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(Nx2) (2,) → (N,) (Nx2) (2,)

$$\frac{\partial J}{\partial \mathbf{w}} = -\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

(Nx1) (Nx2)(2x1) → (2,) (2xN) (N,)

선형회귀 – 입력 차원 (feature dimension) 의 증가 1 → 2

아주대학교 AJOU UNIVERSITY

■ 선형 회귀

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

$$J = \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 = \| \hat{\mathbf{y}} - \mathbf{y} \|_2^2 = \| \mathbf{X}\mathbf{w} - \mathbf{y} \|_2^2$$

$$\frac{\partial J}{\partial \mathbf{w}} = (\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = -\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

```
# Augmented 행렬 X 구성: [1, x]
X = np.hstack([x**0, x])

# 초기 파라미터 (랜덤)
w = np.random.randn(2, 1)

y_hat = X @ w

E = y - y_hat

dJ = -X.T @ E

w  = w - rho * dJ
```

입력 차원이 1일때

$$\hat{y} = w_0x_0 + w_1x_1 + w_2x_2$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(Nx1) (Nx2) (Nx1) (2x1)

$$\underset{\text{(Nx1)}}{\hat{\mathbf{y}}} = \underset{\text{(Nx2) (2x1)}}{\mathbf{X}\mathbf{w}} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

(Nx2) (2x1)

$$\underset{\text{(2x1)}}{\frac{\partial J}{\partial \mathbf{w}}} = \underset{\text{(2xN)}}{-\mathbf{X}^T} \underset{\text{(Nx1)}}{(\mathbf{y} - \underset{\text{(Nx1) (Nx2)(2x1)}}{\mathbf{X}\mathbf{w}})}$$

입력 차원이 2일때

$$\mathbf{X} = \begin{matrix} \mathbf{x_0} & \mathbf{x_1} & \mathbf{x_2} \\ \end{matrix} \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

(Nx3) (Nx1) **(3x1)**

$$\underset{\text{(Nx1)}}{\hat{\mathbf{y}}} = \underset{\textbf{(Nx3)(3x1)}}{\mathbf{X}\mathbf{w}} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

(Nx3) **(3x1)**

$$\underset{\text{(3x1)}}{\frac{\partial J}{\partial \mathbf{w}}} = \underset{\text{(3xN)}}{-\mathbf{X}^T} \underset{\text{(Nx1)}}{(\mathbf{y} - \underset{\text{(Nx1) (Nx3)(3x1)}}{\mathbf{X}\mathbf{w}})}$$